ASan Replication: Stage 2

Broader workloads, larger inputs and repeated measurement

Owner: Zhou Xiaozhou, 1240025753. Other member contributions are recorded separately.

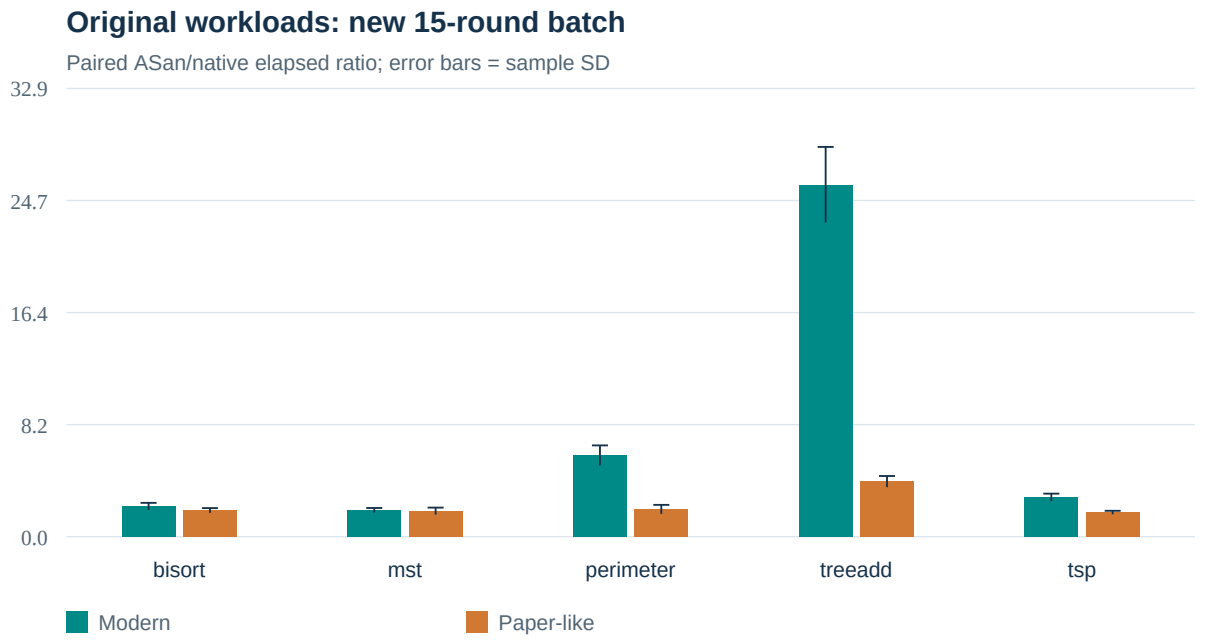
This stage adds Bullet (C++ physics), mafft (sequence processing), and MiBench SHA (hashing), while retaining the five Olden programs. Four larger inputs are distinct workloads, never pooled with reference inputs.

Item	Recorded value
Formal matrix	12 workloads x 3 configurations x 15 rounds = 540
Excluded setup measurements	36 pilot trials + 72 warm-up trials
Validation	45 correctness; 24 upstream + 12 GCC differential
Compiler / source	Clang 14 / llvm-test-suite d52a93a8
Machine	WSL2 Ubuntu 22.04; ~6 GiB; one pinned Linux logical CPU
Randomization	Seed 340; shuffle workloads and configurations within each round
Timeout / builds	600 seconds per trial; build parallelism 2

ASan instruments memory accesses and uses shadow memory, redzones and quarantine to detect classes of memory errors. Its cost must be compared at fixed workload and input.

Record: 20261105T142515Z-caf43a

1. Original programs: a new batch



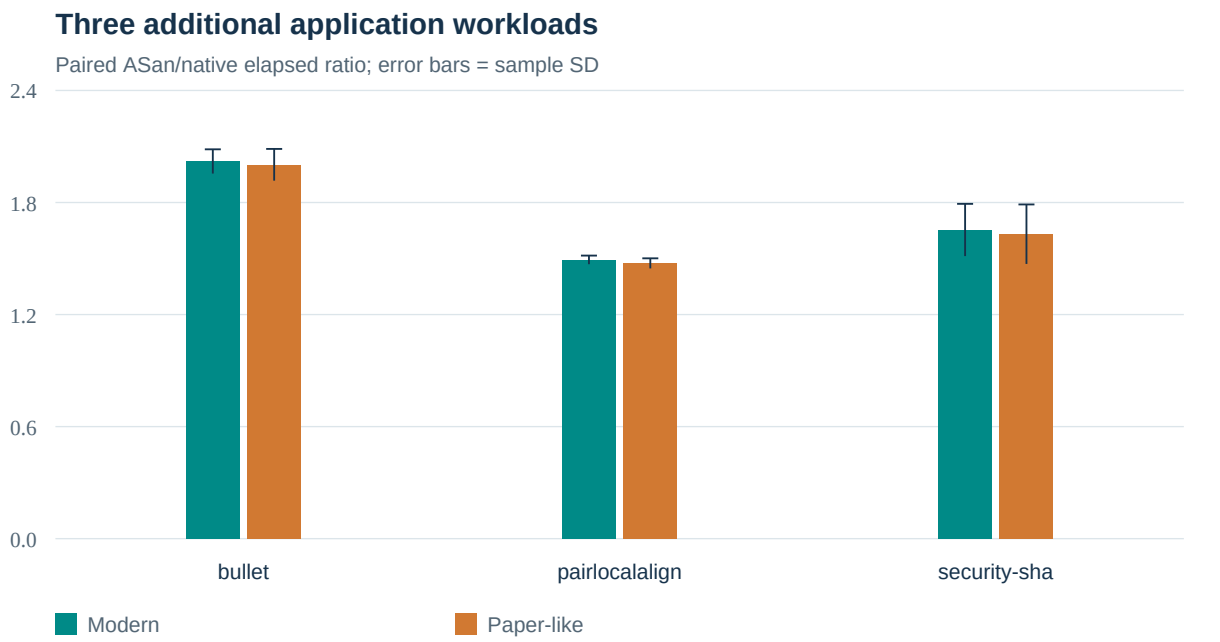
Program	Native s	Modern ratio	Paper-like ratio
bisort	0.260 +/- 0.019	2.222 +/- 0.256	1.919 +/- 0.175
mst	0.031 +/- 0.004	1.936 +/- 0.166	1.870 +/- 0.260
perimeter	0.102 +/- 0.015	5.961 +/- 0.733	2.001 +/- 0.331
treeadd	0.112 +/- 0.021	25.818 +/- 2.773	4.041 +/- 0.410
tsp	0.297 +/- 0.063	2.882 +/- 0.273	1.769 +/- 0.127

Cells show mean +/- sample SD, n=15. ASan/native ratios are computed inside each round before aggregation; ratios of mean times are not substituted. Error bars do not establish significance.

Program	Stage 1 modern (n=5)	Stage 2 modern (n=15)
bisort	2.447 +/- 0.496	2.222 +/- 0.256
mst	1.884 +/- 0.251	1.936 +/- 0.166
perimeter	6.000 +/- 0.783	5.961 +/- 0.733
treeadd	24.728 +/- 5.356	25.818 +/- 2.773
tsp	3.260 +/- 0.288	2.882 +/- 0.273

Batches differ in date, host state and launch wrapper. Stage 1 uses pipe capture; Stage 2 adds a shell, blocking waits and ext4 capture. Cross-batch differences cannot be attributed solely to increasing repetitions. Short inputs remain sensitive to fixed launch costs.

2. Three additional workloads



Program	Native s	Modern ratio	Paper-like ratio
bullet	2.033 +/- 0.053	2.041 +/- 0.066	2.023 +/- 0.086
pairlocalalign	9.394 +/- 0.167	1.508 +/- 0.023	1.489 +/- 0.027
security-sha	0.011 +/- 0.001	1.670 +/- 0.141	1.647 +/- 0.161

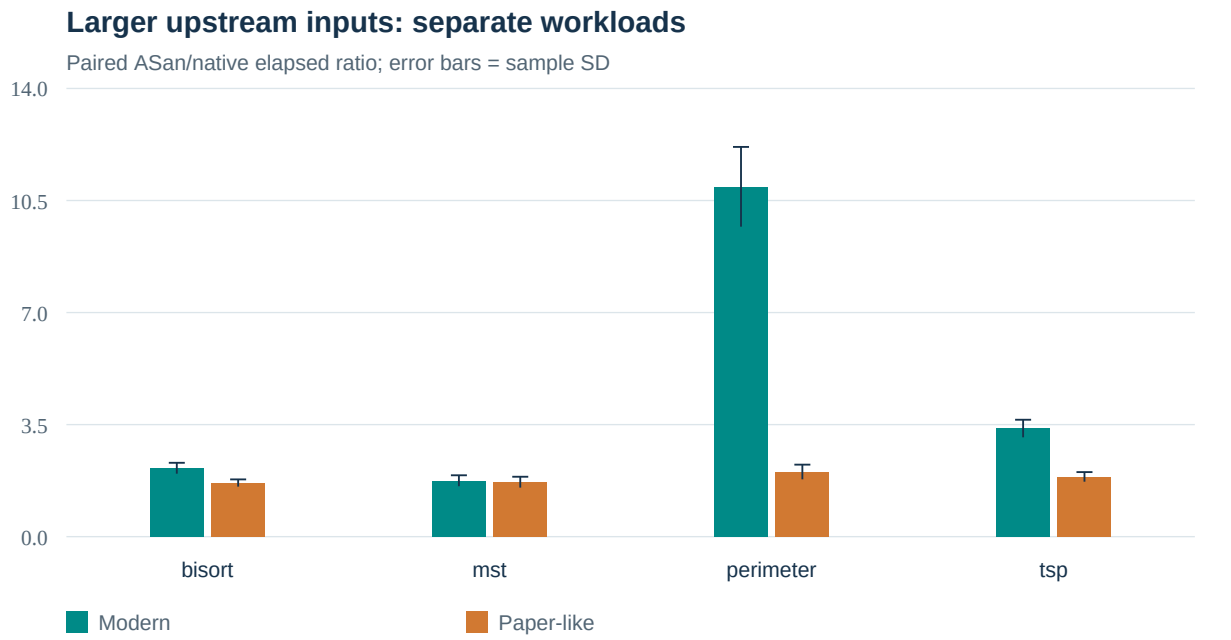
New programs use the same pinned upstream revision without source edits. Commands come from generated .test files; mafft stdin redirection and upstream output-hash verification are preserved. Every reference-size runtime profile passes its upstream reference check.

Bullet adds C++ simulation, mafft adds sequence computation, and SHA adds integer hashing. These improve workload diversity, but this deliberately selected small set is not representative of all C/C++ applications.

Modern ASan ratios in this batch are Bullet 2.041 +/- 0.066, mafft 1.508 +/- 0.023, and SHA 1.670 +/- 0.141. These describe three workloads, rather than a SPEC aggregate score.

Differences between program ratios are observations. Allocation rates, instruction counts and hardware events were not collected, so no particular internal mechanism is established as their cause.

3. Larger inputs: separate comparisons



Program / input	Native s	Modern ratio	Paper-like ratio
bisort / 3000000	1.605 +/- 0.076	2.134 +/- 0.170	1.673 +/- 0.113
mst / 4000	0.853 +/- 0.100	1.743 +/- 0.171	1.699 +/- 0.170
perimeter / 11	0.387 +/- 0.060	10.911 +/- 1.244	2.017 +/- 0.229
tsp / 2048000	0.573 +/- 0.037	3.372 +/- 0.274	1.864 +/- 0.150

Program	Ref native s	Large native s
bisort	0.260 +/- 0.019	1.605 +/- 0.076
mst	0.031 +/- 0.004	0.853 +/- 0.100
perimeter	0.102 +/- 0.015	0.387 +/- 0.060
tsp	0.297 +/- 0.063	0.573 +/- 0.037

Larger arguments use upstream LARGE_PROBLEM_SIZE settings, but the pinned revision still supplies reference-size expected outputs. Large outputs are therefore checked against an independent GCC build using generated build-tree expectations. Upstream sources and original references remain unchanged. This is cross-compiler consistency, not an independent algorithmic proof.

For example, perimeter modern mean ratio changes from 5.961 at the reference input to 10.911 at the larger input. Larger inputs need not reduce ratios; these data establish neither a cause nor a universal trend. treeadd input is unchanged.

4. Peak RSS and short workloads

Workload	Native MiB	Modern MiB	Paper-like MiB
bisort-ref	17.84	131.49	40.47
mst-ref	26.36	36.53	36.21
perimeter-ref	86.86	282.34	124.60
treeadd-ref	129.48	1088.45	292.48
tsp-ref	66.07	316.79	112.79
bullet-ref	56.73	175.01	67.87
pairlocalalign-ref	140.15	221.42	194.04
security-sha-ref	3.04	4.69	4.69
bisort-large	65.83	532.48	148.46
mst-large	399.23	514.11	510.07
perimeter-large	342.86	1146.32	484.61
tsp-large	130.10	640.79	220.79

Values are GNU time maximum resident-set means. RSS measures resident memory; VmPeak measures peak virtual address space. These results do not reproduce the paper's 3.37x VmPeak result.

Native mean below 0.5 seconds (warning threshold, never an exclusion rule): bisort-ref, mst-ref, perimeter-ref, treeadd-ref, tsp-ref, security-sha-ref, perimeter-large

Timing includes taskset, GNU time, shell, launch and exit. Complete streams are captured on Linux ext4; lossless compression and copying happen after timing. Decompression restores all bytes. No formal samples are removed or estimated startup costs subtracted. More rounds describe variation but do not remove systematic bias.

5. Audit, reproduction and limits

An independent read-only audit checks the complete matrix, randomized schedule, CSV/JSON, stdout digests, raw GNU time metrics, means and sample SDs. All formal trials exit successfully; 25 injected-error trials return 1 and 20 controls return 0.

An earlier complete matrix was excluded after Windows idle standby. The accepted batch uses process-scoped idle-sleep prevention and checks host sleep events at completion, without changing system power policy. Development and excluded records are archived separately and supply no report numbers.

The final host query initially failed because Windows PowerShell modules could not load through WSL. The same interval was reviewed from the Windows host. Before completing acceptance metadata, all prior hashes were checked; samples and protocol remained byte-identical. The earlier status and manifest are retained.

Audit	Result
Formal / correctness	540 / 45 passed
Upstream / GCC differential checks	24 / 12 passed
Manifest SHA-256 checks	2263 matched
Original source	Pinned revision; git diff clean before and after
Preservation	Stage 1 record and reports retained independently

Reproduce

```
.\scripts\Run-Stage2.ps1 -PrecheckOnly
.\scripts\Run-Stage2.ps1
python scripts/verify_stage2.py
python scripts/make_stage2_reports.py
```

Conclusions are restricted to these programs, inputs, profiles and machine. Modern and paper-like settings differ in redzones, quarantine and allocation-stack capture. This stage performs no ablation and cannot isolate individual parameter contributions.

SPEC CPU2006, writes-only instrumentation and Chromium bug discovery are untested. Passing finite controls does not prove zero false positives. WSL core mapping, temperature, power policy and background load remain uncontrolled.

Sources

Serebryany et al., AddressSanitizer: A Fast Address Sanity Checker, USENIX ATC 2012. https://www.usenix.org/system/files/conference/atc12/atc12-final39.pdf

LLVM test-suite d52a93a891cde7a76aca63f7a201c5a5fe459f41; generated tests, protocol.json, measurements.json, summary.json and raw logs in the evidence directory.

AI usage: AI assisted implementation, execution, verification, analysis and drafting. Numerical results come from saved local runs. Planned work is not recorded as completed contribution, and this report does not claim other members' work.